

IA Aplicada com LLMs

Aula 02 — O painel de controle da chamada

Onde paramos

O deck anterior escolheu o **motor**.

Este é sobre o **painel de controle**: os parâmetros que mudam **como** o modelo escolhe cada token.

Você já viu dois deles na Aula 01 — `temperature` e `top_p` .

O erro característico de quem começa

Tratar parâmetro como mágica:

- O texto repetiu? *Aumenta a penalidade.*
- Alucinou? *Baixa a temperatura.*
- Não seguiu o formato? *Mexe no `top_p`.*

Quase sempre errado.

A maior parte dos problemas de saída de LLM é problema de **prompt** ou de **arquitetura** — e girar botões sem medir **esconde** a causa.

Roteiro

1. **Recapitulação** — logits, softmax, temperatura, `top_p`
2. **Parâmetro é contrato de API** — a lição que economiza mais depuração
3. **Anatomia da requisição** — o painel completo
4. **Os parâmetros novos** — `seed`, `n`, `stream`
5. **Determinismo** — medindo o que a Aula 01 afirmou
6. **Saída estruturada** — e por que é a base do *tool calling*

Objetivos

- **Recuperar** o efeito de `temperature` e `top_p` na distribuição
- **Explicar** por que parâmetro é **contrato de API**
- **Detectar truncamento** pelo `finish_reason`
- **Demonstrar** que `temperature=0` não garante saída idêntica
- **Obter JSON confiável** com `response_format` e JSON Schema
- **Escolher** parâmetros a partir do tipo de tarefa

Parte 1

Recapitulando a Aula 01

Logits → softmax → temperatura

A última camada produz um número real por token do vocabulário: os **logits**. Não são probabilidades (podem ser negativos, não somam 1).

Num vocabulário de 32 mil tokens, são **32 mil logits por posição**.

$$P(\text{token}_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

onde z_i é o **logit** do token i e T a temperatura.

z é a notação de logit em **todas** as fórmulas desta aula.

O efeito, em números

Completando "O céu estava..." com logits [3.2, 2.1, 1.0, 0.2] :

T	azul	claro	cinza	roxo
0.1	100.0%	0.0%	0.0%	0.0%
0.5	88.8%	9.8%	1.1%	0.2%
1.0	67.0%	22.3%	7.4%	3.3%
1.5	54.2%	26.0%	12.5%	7.3%
2.0	46.9%	27.0%	15.6%	10.5%

Os logits não mudaram. A temperatura não altera o que o modelo "pensa" — altera o quanto ele **arrisca** ao escolher.

Três conclusões que sustentam o resto

1. **Temperatura reformata a distribuição.** $T < 1$ concentra, $T > 1$ achata, $T \rightarrow 0$ vira `argmax`
2. `top_p` corta a cauda de forma **adaptativa** (mantém tokens até acumular p da massa). `top_k` faz igual com número fixo — pior por ser fixo. **Ajuste um, não os dois**
3. `temperature=0` **não garante saída idêntica** — a Parte 5 mede

Parte 2

Parâmetro é contrato de API

A lição que economiza mais depuração

Os parâmetros existem no **runtime de inferência**. O que chega até você é o subconjunto que **o provedor** decidiu expor, com os nomes **dele**.

Situação	Exemplo real	Como você descobre
não existe na API	<code>top_k</code> existe no Ollama e no vLLM; o <code>chat/completions</code> da Mistral não o expõe	lendo a documentação
tem outro nome	OpenAI: <code>seed</code> · Mistral: <code>random_seed</code>	erro 422 — ou silêncio
é aceito e ignorado	campos desconhecidos descartados	você não descobre

A terceira é a perigosa

Você acha que configurou algo.

O número está lá, no seu código.

O comportamento não muda.

E você atribui isso **ao modelo**.

A válvula de escape

```
resposta = client.chat.completions.create(  
    model="mistral-small-latest",  
    messages=[...],  
    temperature=0,  
    extra_body={"random_seed": 42},    # campo da Mistral  
    )                                  # que o SDK não conhece
```

Ao trocar de provedor, revalide os parâmetros.

Portabilidade da *chamada* não é portabilidade do *comportamento*.

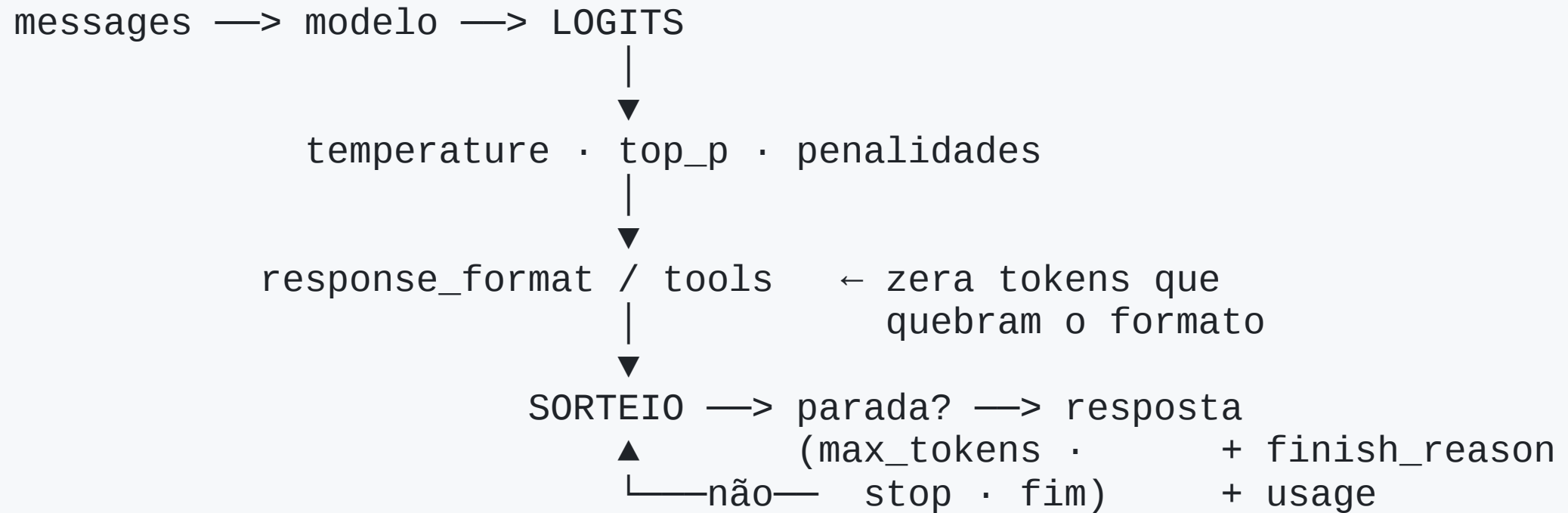
Parte 3

Anatomia da requisição

O painel completo

Campo	O que faz	Onde age
<code>temperature</code>	achata/aguça a distribuição	amostragem
<code>top_p</code>	corta a cauda por massa acumulada	amostragem
<code>frequency_penalty</code> / <code>presence_penalty</code>	descontam logits já usados	amostragem
<code>seed</code> / <code>random_seed</code>	fixa o gerador (<i>best effort</i>)	amostragem
<code>max_tokens</code> · <code>stop</code>	teto e marca de parada	parada
<code>response_format</code>	impõe formato de saída	decodificação restrita

Onde cada botão age



`response_format` age **depois** das penalidades e **antes** do sorteio.

Por isso o JSON Schema funciona

Ele **não pede** educadamente que a saída seja JSON.

Ele torna **impossível** sortear um token que quebre o formato.

(Detalhe na Parte 6.)

Parte 4

Os parâmetros que a Aula 01 não viu

`max_tokens` e `finish_reason`

`max_tokens` = teto de tokens **gerados**. Controla custo, latência e geração desgovernada.

O problema: quando o teto é atingido, a geração **para no meio** — e a resposta chega como se estivesse completa.

<code>finish_reason</code>	Significado
<code>"stop"</code>	terminou por conta própria (ou bateu no <code>stop</code>)
<code>"length"</code>	bateu no <code>max_tokens</code> — TRUNCOU
<code>"tool_calls"</code>	pediu uma ferramenta (Aula 03)
<code>"content_filter"</code>	bloqueado por política

Tratar isso é obrigatório

```
escolha = resposta.choices[0]

if escolha.finish_reason == "length":
    raise ValueError("resposta truncada")

conteudo = escolha.message.content
```

Tão automático quanto checar o *status code* de uma requisição HTTP.

O que nunca se faz: seguir em frente fingindo que a resposta está completa. É assim que um JSON pela metade chega no banco.

(Dimensionamento e detalhes: deck 03.)

stop: cortar na sua marca

`max_tokens` corta por **orçamento**. `stop` corta por **conteúdo**.

"pare depois de 200 tokens" × "pare quando chegar na despedida"

A sequência **não aparece** na saída.

```
stop=["Usuário:"]    # impede o modelo de inventar  
                    # a fala do outro lado
```

(Formatos e cuidados: deck 03.)

seed : reprodutibilidade *best effort*

Fixa a semente do gerador pseudoaleatório.

Mesma semente + mesmo prompt + mesmo modelo → **tende** a sair igual.

Tende, não garante (Parte 5).

Use para **depurar** um caso estranho. **Nunca** como base de teste automatizado.

As penalidades, em uma linha

$$z'_i = z_i - \underbrace{\alpha \cdot c_i}_{\text{frequency}} - \underbrace{\beta \cdot 1[c_i > 0]}_{\text{presence}}$$

- `frequency_penalty` cresce com a **contagem** → *frequência controla o texto*
- `presence_penalty` desconto **fixo** na reincidência → *presença controla o assunto*

Faixa típica: -2 a 2. **Negativo encoraja** repetição.

⚠ Em saída estruturada, penalidade é **ativamente perigosa**.

(Comparação numérica e quando não usar: deck 03.)

n: várias respostas de uma vez

n=5 → cinco respostas independentes, numa requisição.

Você paga o **prompt uma vez** e as saídas cinco vezes — mais barato que cinco chamadas.

Serve para: comparar variação, oferecer alternativas, e **self-consistency** (gerar N raciocínios e ficar com a resposta majoritária) — Aula 03.

stream: muda o produto, não o desempenho

```
fluxo = client.chat.completions.create(..., stream=True)
for evento in fluxo:
    if evento.choices and evento.choices[0].delta.content:
        print(evento.choices[0].delta.content, end="", flush=True)
```

O tempo total não muda — o modelo não gera mais rápido.

Muda o **TTFT percebido**: a espera deixa de parecer travamento.

Custo no código: não há resposta inteira para validar antes de mostrar, e a contagem de tokens vem no último evento — **quando vem**.

Interação entre parâmetros

Combinação	O que acontece
<code>temperature</code> alta + <code>top_p</code> baixo	brigam entre si — por isso se ajusta um
penalidades + <code>temperature=0</code>	determinístico, mas os logits seguem deformados
<code>max_tokens</code> curto + JSON longo	trunca no meio → <code>length</code> e saída inválida
penalidades + saída estruturada	penaliza as chaves do JSON → deixe em 0

Parte 5

Determinismo

temperature=0 não é determinismo

A Aula 01 afirmou. Agora medimos: 10 chamadas idênticas, contando saídas distintas.

Por que varia:

- o provedor agrupa requisições em **lotes de tamanho variável**, e soma de ponto flutuante **não é associativa** — $(a + b) + c \neq a + (b + c)$
- uma diferença no último dígito às vezes troca o **argmax**, e **um token diferente muda toda a continuação**
- **MoE**: o roteamento é sensível ao lote
- **alias** **-latest** pode ter mudado desde ontem

A consequência de engenharia

```
# RUIM – vai falhar sem que nada esteja quebrado
assert resposta == "A capital da França é Paris."

# BOM – testes por PROPRIEDADE
assert "paris" in resposta.lower()
assert len(resposta) < 200
dados = json.loads(resposta)
assert set(dados) == {"nome", "idade", "cidade"}
assert re.fullmatch(r"\d{3}\.\d{3}\.\d{3}-\d{2}", dados["cpf"])
```

Você não testa o **texto** que o LLM produziu.

Testa o **que ele precisa satisfazer**.

É o germe dos **evals**, no fim do curso.

Parte 6

Saída estruturada

O problema

Você pede JSON no prompt. Funciona 9 vezes em 10. Na décima:

```
Claro! Aqui está o JSON com os dados extraídos:
```

```
```json
{"nome": "Ana Souza", "idade": 34}
```
```

```
Precisa de mais alguma coisa?
```

E quando não é o embrulho, o modelo chamou o campo de `nome_completo`, ou devolveu `"idade": "34"` (string).

Texto livre não tem contrato. Sistema sobre texto livre é sistema sem tipos — e você descobre isso em produção.

Três estratégias, garantia crescente

| | Como | Garante |
|----------|--|-----------------------------------|
| A | só prompt: "responda em JSON" | nada |
| B | <code>response_format={"type": "json_object"}</code> | é JSON válido |
| C | <code>response_format</code> com JSON Schema | JSON válido no seu formato |

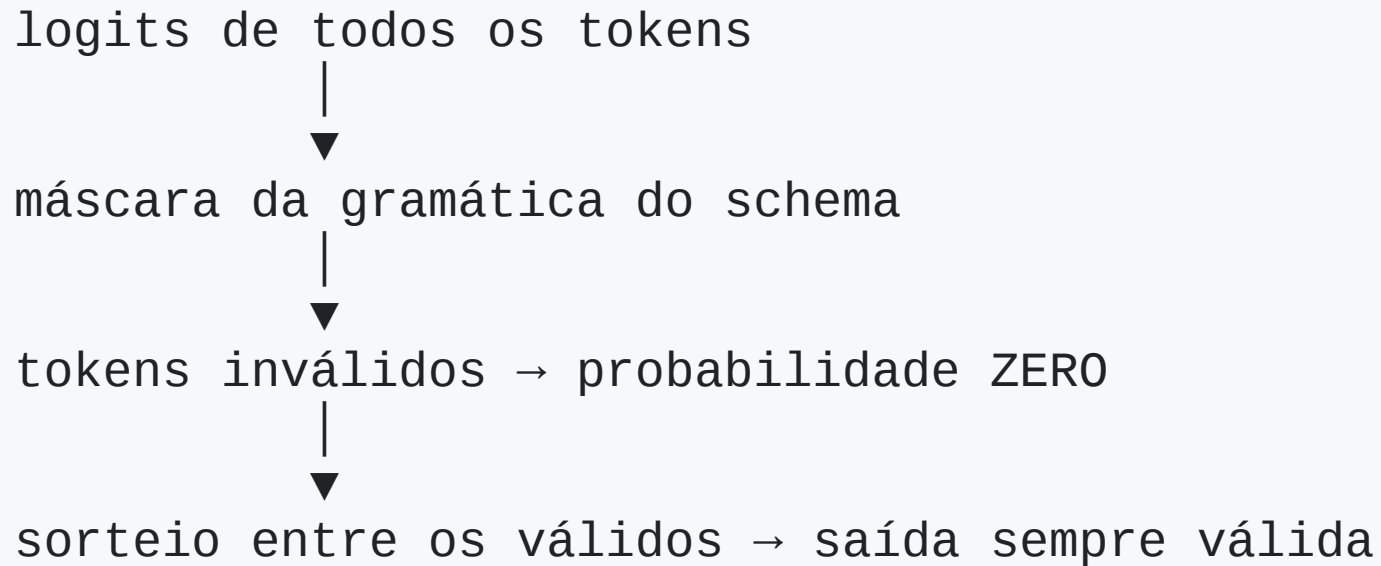
B resolve o embrulho e o "Claro! Aqui está".

B **não** resolve *qual* JSON — as chaves ainda são as que o modelo quiser.

A estratégia C

```
SCHEMA = {  
    "type": "object",  
    "properties": {  
        "nome": {"type": "string"},  
        "idade": {"type": "integer"},  
        "cidade": {"type": "string"},  
    },  
    "required": ["nome", "idade", "cidade"],  
    "additionalProperties": False,  
}  
  
response_format={  
    "type": "json_schema",  
    "json_schema": {"name": "cadastro", "schema": SCHEMA, "strict": True},  
}
```

Por que a decodificação restrita funciona



Se o schema diz que depois de `{"idade":` vem inteiro, os tokens de aspas **não podem** ser sorteados. Probabilidade **zero**, não "baixa".

O formato deixa de ser um pedido e vira **propriedade da geração**.

O que o schema NÃO garante

```
{"nome": "Ana Souza", "idade": 134, "cidade": "Campinas"}
```

Passa em **qualquer** validação de schema. A idade está errada.

Formato válido com conteúdo errado é o erro mais perigoso de todos —
atravessa todas as suas validações sem disparar nada.

Defesas: `enum` para categorias, `minimum` / `maximum`, `pattern`; validar regra de negócio depois; conferir se o valor **aparece no texto de origem**.

O gancho: tool calling é isto

```
tools = [{  
  "type": "function",  
  "function": {  
    "name": "consultar_saldo",  
    "parameters": {  
      "type": "object",  
      "properties": {"conta": {"type": "string"}},  
      "required": ["conta"],  
    },  
  },  
}]
```

<- é um JSON Schema

O modelo **não executa** nada: gera um **JSON válido** dizendo qual função quer, e devolve `finish_reason="tool_calls"`. Quem executa é o seu programa.

Sem saída estruturada confiável não existe agente confiável.

Cartão de referência

| Tarefa | temperature | Penalidades | Saída |
|--------------------------|-------------|-----------------|----------------|
| Extração / classificação | 0 | 0 | JSON Schema |
| Chamada de ferramenta | 0 – 0,2 | 0 | tools |
| Código | 0,2 – 0,5 | 0 | texto / schema |
| Resposta factual, RAG | 0,2 – 0,5 | 0 | texto |
| Redação, explicação | 0,7 – 1,0 | 0 | texto |
| Brainstorm | 1,0 – 1,3 | testar presence | texto |

Temperatura baixa **não** cura alucinação — *grounding* cura.

Três regras que valem mais que a tabela

1. **Mexa em um parâmetro de cada vez** e meça. Dois de uma vez e você não sabe qual causou o quê
2. **temperature=0** não é "resposta correta" — é "resposta mais provável".
O modelo pode estar confiantemente errado, e a temperatura zero só garante que ele **erre sempre igual**
3. **Se você está girando botões há meia hora**, o problema não é o botão.
É o prompt, o modelo, ou a arquitetura

Síntese

- Todo o painel age sobre a distribuição do **próximo token**
- **Parâmetro é contrato de API**: pode não existir, ter outro nome, ou ser **aceito e ignorado em silêncio** — o caso traiçoeiro
- `finish_reason="length"` = resposta truncada. Checar é obrigatório
- `seed` é *best effort*: depuração, nunca teste
- `stream` não muda o tempo total, muda o **TTFT percebido**
- `temperature=0` **não é determinismo** → teste **propriedades**, não igualdade
- **Saída estruturada** em 3 níveis; o Schema zera os tokens que quebrariam o formato
- **Schema válido ≠ dado correto**
- **Tool calling é saída estruturada** com o schema da função